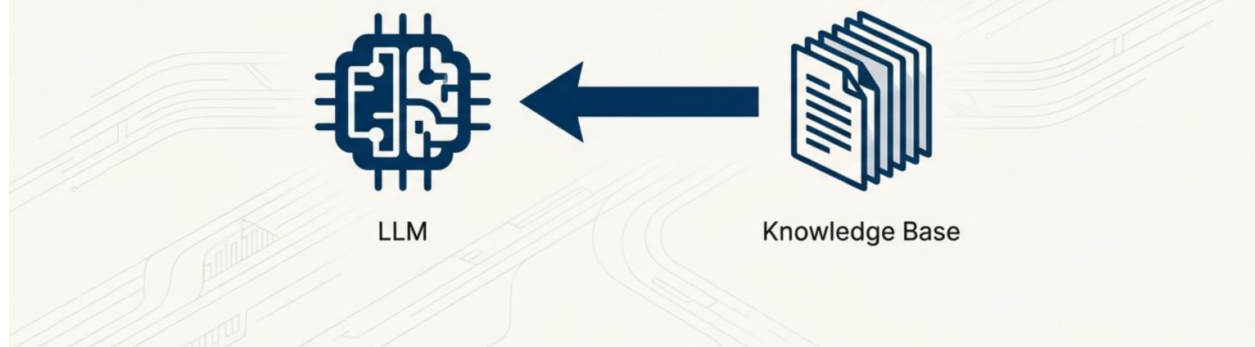


RAG(검색 증강 생성) 기반 문서 분석 챗봇 구축

신뢰할 수 있는 엔터프라이즈 AI의 핵심, RAG

대규모 언어 모델의 한계를 넘어, 검증 가능한 지식 기반 챗봇 구축의 이론과 실제

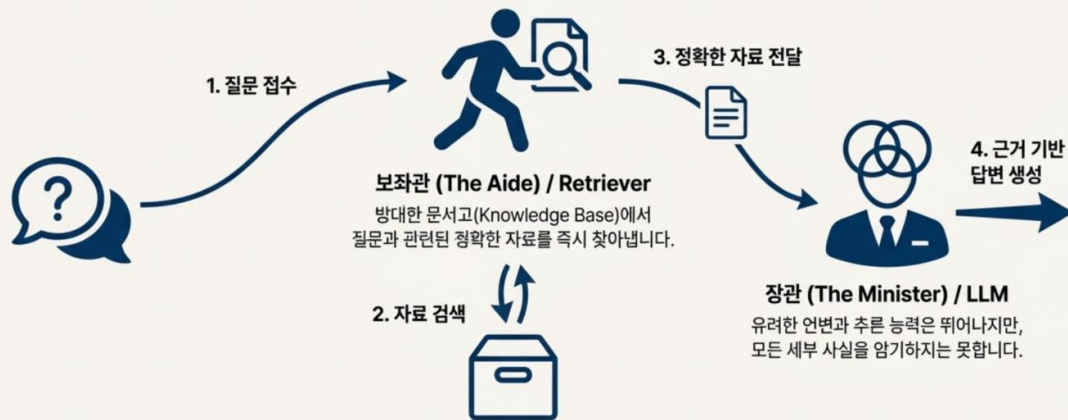


서론: 대규모 언어 모델의 인식론적 한계와 RAG의 필연성

인공지능 기술의 급격한 발전, 특히 2025년을 기점으로 한 생성형 AI(Generative AI)의 엔터프라이즈 도입 가속화는 우리에게 새로운 기술적 패러다임을 제시하고 있습니다. GPT-4o와 같은 최신 대규모 언어 모델(Large Language Models, LLMs)은 인간에 버금가는 유창한 언어 구사 능력을 보여주고 있지만, 이들은 본질적으로 '확률론적 앵무새(Stochastic Parrots)'라는 구조적 한계를 안고 있습니다. 이러한 LLM의 한계를 극복하고, 기업 및 공공 기관의 신뢰할 수 있는 지식 관리 시스템을 구축하기 위한 핵심 기술인 **검색 증강 생성(Retrieval-Augmented Generation, RAG)**의 이론적 배경부터 구현, 그리고 배포까지의 전 과정을 심도 있게 다룹니다.

우리는 먼저 LLM이 가진 기억의 본질, 즉 **매개변수적 메모리(Parametric Memory)**의 정적인 특성을 이해해야 합니다. 모델은 학습 시점의 데이터에 갇혀 있으며, 이는 마치 2023년에 세상과 단절된 백과사전 편집자와 같습니다. 2024년 9월에 출시된 아이폰 16의 스펙이나, 지난주에 발표된 서울시의 새로운 도시 계획 조례에 대해 이 모델에게 묻는다면, 모델은 침묵하거나 혹은 더욱 위험하게도 그럴듯한 거짓말을 만들어내는 **환각(Hallucination)** 현상을 일으킵니다.

해결책: 천재 장관(LLM)에게 유능한 보좌관(Retriever)을 붙여주다



이 둘의 협업이 바로 RAG의 본질입니다. 장관은 보좌관이 찾아준 자료를 '근거'로 삼아 답변을 생성합니다.

이러한 문제를 해결하기 위해 "**장관과 보좌관**"이라는 강력한 은유를 통해 RAG 시스템을 설명합니다. 여기서 장관(LLM)은 유려한 언변과 추론 능력을 갖추었으나 세부적인 팩트를 모두 암기할 수 없는 의사결정자이며, 보좌관(Retriever)은 방대한 문서고에서 필요한 정보를 순식간에 찾아내는 검색 전문가입니다. 이 둘의 협업이야말로 RAG의 본질이며, 이를 기술적으로 구현하기 위해 우리는 벡터 임베딩, 코사인 유사도, 청킹(Chunking), 그리고 벡터 데이터베이스와 같은 개념들을 수학적 원리와 함께 상세히 분석할 것입니다.

더 나아가, 2025년 현재의 RAG 기술 트렌드를 반영하여 단순한 텍스트 검색을 넘어선 **GraphRAG**, **Agentic RAG**의 개념과, 교육 및 엔터프라이즈 환경에서의 실제 적용 사례, 그리고 환각률을 최소화하기 위한 최신 전략들을 포괄적으로 다룰 것입니다.

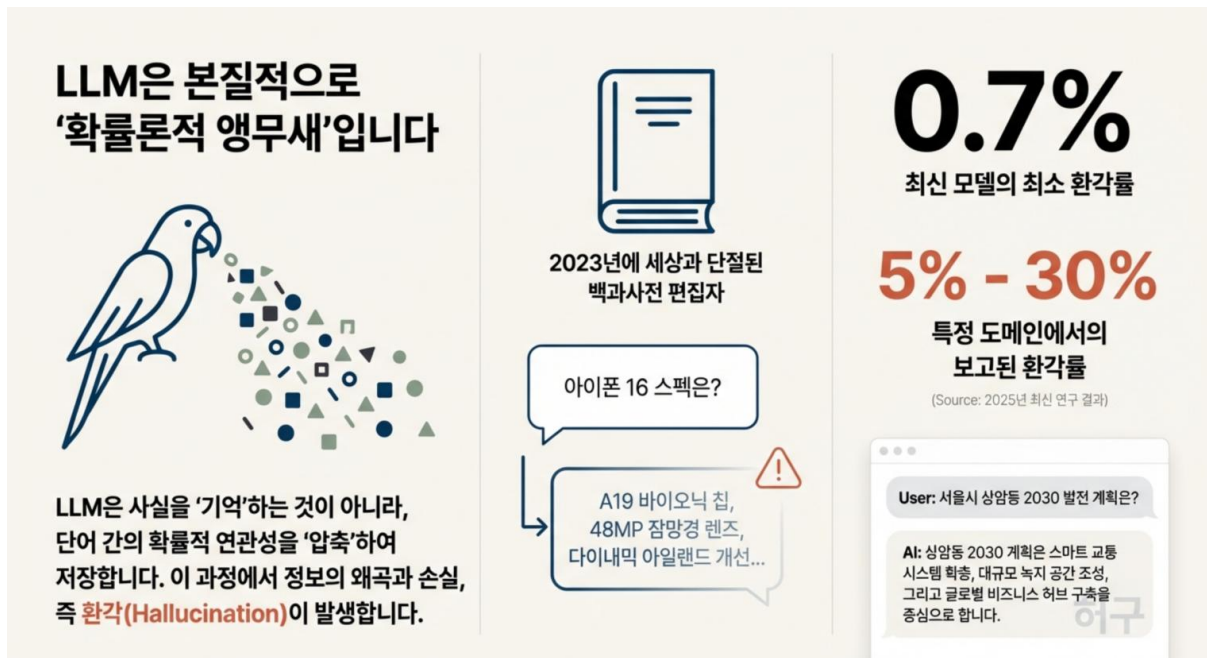
단순한 코딩 튜토리얼을 넘어, AI 엔지니어로서 갖추어야 할 데이터 파이프라인 설계의 철학과 최적화 전략을 함양하는 것을 목표로 합니다.

1 RAG 의 이론적 토대: 벡터 공간과 의미론적 검색

1.1 언어 모델의 작동 원리와 환각의 메커니즘

언어 모델은 근본적으로 '다음에 올 단어 맞추기 게임'을 수행하는 통계 기계입니다. 주어진 텍스트 시퀀스 $w_1, w_2, \dots, w_{t-1}$ 이 있을 때, 다음 단어 w_t 가 등장할 조건부 확률 $P(w_t | w_1, \dots, w_{t-1})$ 을 최대화하는 방향으로 학습됩니다. 이 과정에서 모델은 사실(Fact)을 '기억'하는 것이 아니라, 단어와 단어 사이의 확률적 연관성을 '압축'하여 저장합니다.

이러한 압축 과정은 손실을 동반합니다. JPEG 이미지가 압축될 때 픽셀 정보가 뭉개지듯, LLM 의 지식 압축 과정에서도 정보의 디테일이 소실되거나 왜곡됩니다. 이것이 바로 환각 현상의 원인입니다. 예를 들어, 사용자가 "서울시 상암동의 2030 발전 계획"을 물었을 때, 모델은 '상암동', '발전', '계획'이라는 단어들과 연관된 일반적인 도시 개발 용어(교통 확충, 녹지 조성 등)를 확률적으로 조합하여 답변을 생성합니다. 이 답변은 문법적으로 완벽하고 논리적으로 타당해 보이지만, 실제 서울시의 행정 문서와는 무관한 허구일 가능성이 높습니다.

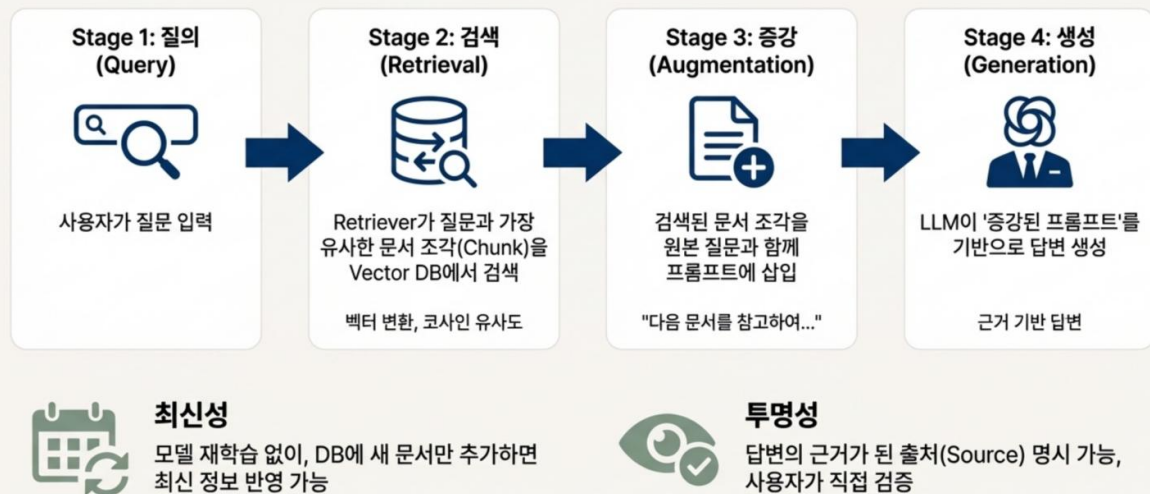


2025 년 최신 연구 결과에 따르면, 최신 모델인 Gemini-2.0-Flash 조차 약 0.7%의 환각률을 보이며, 일반적인 모델들은 특정 도메인 질문에 대해 5%에서 30%까지의 환각률을 보일 수 있다고 보고되고 있습니다. 이는 엔터프라이즈 환경, 특히 법률이나 의료와 같이 정확성이 생명인 분야에서 순수

LLM 을 그대로 사용하는 것이 얼마나 위험한지를 시사합니다. 따라서 우리는 모델의 내부 지식(Parametric Knowledge)에 의존하는 대신, 외부의 신뢰할 수 있는 지식 소스(Non-Parametric Knowledge)를 연결하는 RAG 아키텍처를 도입해야 합니다.

1.2. RAG 아키텍처의 핵심: 검색과 생성의 결합

RAG 아키텍처는 검색과 생성을 결합하여 투명성과 최신성을 확보합니다



RAG 는 생성 모델에 정보 검색(Information Retrieval, IR) 시스템을 결합한 하이브리드 아키텍처입니다. 그 작동 방식은 국정 감사를 받는 장관의 비유를 통해 직관적으로 이해할 수 있습니다. 장관(LLM)이 질문을 받으면, 즉시 답변하는 대신 보좌관(Retriever)에게 관련 자료를 요청합니다. 보좌관은 수천 페이지의 보고서(Knowledge Base) 중에서 질문과 가장 관련된 페이지(Relevant Chunks)를 찾아 장관에게 전달합니다. 장관은 이 자료를 바탕으로 답변을 구성합니다.

이 과정은 기술적으로 다음과 같은 흐름을 가집니다:

- 질의(Query):** 사용자가 질문을 입력합니다.
- 검색(Retrieval):** 리트리버가 질문을 벡터로 변환하고, 벡터 데이터베이스에서 가장 유사한 문서 조각(Chunk)을 찾습니다.
- 증강(Augmentation):** 찾아낸 문서 조각을 프롬프트에 포함시켜 질문과 함께 LLM 에 전달합니다. ("다음 문서를 참고하여 질문에 답하시오: [문서 내용]")
- 생성(Generation):** LLM 이 제공된 문서를 근거로 답변을 생성합니다.

이 구조의 가장 큰 장점은 **최신성**과 **투명성**입니다. 모델을 재학습시키지 않고도 데이터베이스에 새로운 문서만 추가하면 AI가 최신 정보를 알게 됩니다. 또한, 답변 생성에 사용된 문서의 출처를 명시할 수 있어 사용자가 정보의 진위 여부를 검증할 수 있게 됩니다.

1.3. 청킹(Chunking): 데이터를 AI가 소화할 수 있는 크기로

수백 페이지에 달하는 PDF 문서를 LLM에 한 번에 입력하는 것은 비효율적일 뿐만 아니라 기술적으로도 불가능한 경우가 많습니다. LLM은 한 번에 처리할 수 있는 정보의 양, 즉 **컨텍스트 윈도우(Context Window)**에 제한이 있기 때문입니다. 비록 GPT-4 Turbo나 Gemini 1.5 Pro와 같은 최신 모델들이 수십만 토큰의 컨텍스트 윈도우를 제공한다 하더라도, 방대한 문서를 모두 입력하는 것은 막대한 비용(토큰당 과금)과 처리 지연(Latency)을 유발합니다. 또한, 'Lost in the Middle' 현상으로 인해, 입력된 텍스트의 중간 부분에 위치한 정보는 모델이 잘 집중하지 못하는 경향이 있습니다.

따라서 우리는 문서를 **청크(Chunk)**라고 불리는 작은 조각으로 나누어야 합니다. 이 과정을 **청킹(Chunking)**이라고 합니다. 청킹은 단순히 문서를 자르는 것이 아니라, 의미의 단위로 분절하는 예술에 가깝습니다.

1단계: 지식 준비 - AI가 소화할 수 있도록 문서를 '청크'로 분할합니다

Why Chunking is Necessary

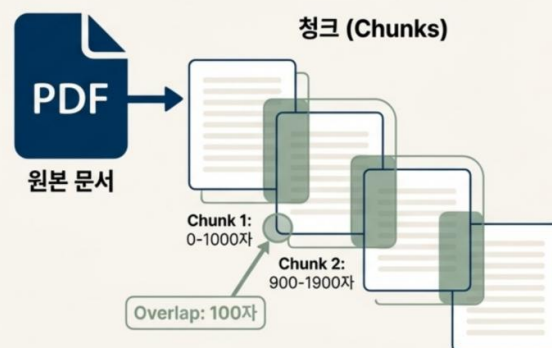
- **컨텍스트 윈도우 (Context Window) 한계:** LLM은 한 번에 처리할 수 있는 정보의 양이 제한적입니다.
- **비용 및 지연 (Cost & Latency):** 전체 문서를 입력하면 토큰 비용과 처리 시간이 급증합니다.
- **'Lost in the Middle' 현상:** 긴 텍스트의 중간 정보는 LLM이 놓치기 쉽습니다.

The Trade-off of Chunking

- **작은 청크 (100-200 토큰):** 구체적 사실 검색에 유리하지만, 문맥이 손실될 위험이 있습니다.
- **큰 청크 (1000-2000 토큰):** 문맥은 풍부하지만, 불필요한 정보(Noise)가 많아 검색 정확도가 저하될 수 있습니다.

The Solution: Overlap

- 청크 간에 내용을 겹치게 하여(예: 100자), 문장이 경계에서 잘려 의미가 단절되는 것을 방지합니다.



1.3.1 청크 크기와 오버랩의 트레이드오프 (Trade-off)

청크의 크기를 결정하는 것은 RAG 성능 최적화의 핵심입니다.

- **작은 청크 (예: 100-200 토큰):** 구체적인 사실 관계를 검색하는 데 유리합니다. 하지만 문맥이 잘려 나갈 위험이 있어, 모델이 "그것은 5% 증가했다"라는 문장은 찾았지만, 무엇이 증가했는지는 모르는 상황이 발생할 수 있습니다.
- **큰 청크 (예: 1000-2000 토큰):** 충분한 문맥을 포함하므로 복잡한 추론이 필요한 질문에 적합합니다. 그러나 불필요한 정보(Noise)가 많이 포함되어 검색 정확도가 떨어지거나 비용이 증가할 수 있습니다.⁹

이를 보완하기 위해 **오버랩(Overlap)** 기법을 사용합니다. 예를 들어 1000 자 단위로 자를 때, 앞 청크의 마지막 100 자를 뒤 청크의 시작 부분에 중복시킵니다. 이는 마치 드라마의 다음 화가 시작될 때 "지난 이야기"를 보여주는 것과 같아서, 문장이 청크 경계에서 잘리더라도 의미가 단절되는 것을 방지합니다.

1.4. 벡터 임베딩과 코사인 유사도: 의미를 숫자로 변환하다

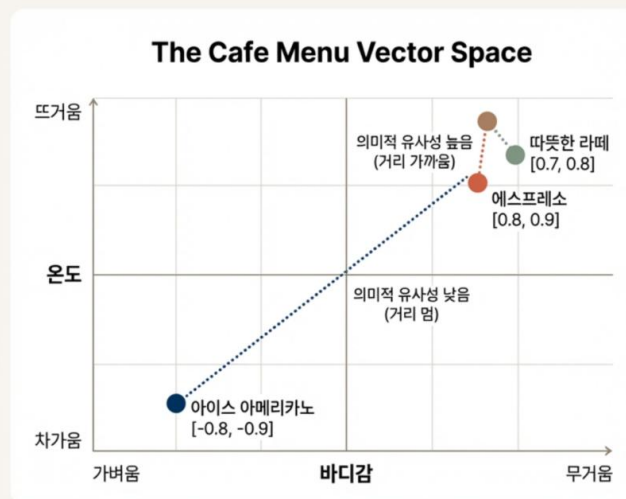
컴퓨터는 '사랑', '배신', '커피'와 같은 단어의 의미를 직관적으로 이해하지 못합니다. RAG 시스템이 질문과 관련된 문서를 찾기 위해서는 텍스트를 컴퓨터가 계산할 수 있는 숫자들의 나열, 즉 **벡터(Vector)**로 변환해야 합니다. 이 과정을 **임베딩(Embedding)**이라고 합니다.

1.4.1 벡터 공간의 직관적 이해: 카페 메뉴 예시

2단계: 의미의 좌표화 - 텍스트를 벡터 공간의 한 점으로 변환합니다

컴퓨터는 '사랑'이나 '커피' 같은 단어의 의미를 이해하지 못합니다.
임베딩(Embedding)은 텍스트를 컴퓨터가 계산할 수 있는 숫자 배열, 즉 **벡터(Vector)**로 변환하는 과정입니다.

벡터 공간에서 거리가 가까울수록 의미적으로 유사합니다. 실제 임베딩 모델은 1536차원 이상의 초고차원 공간을 사용하여 미세한 뉘앙스까지 포착합니다.



카페 음료를 2 차원 벡터 공간에 표현하는 훌륭한 예시를 제시합니다.

- X축: 바디감 (가벼움 ↔ 무거움)
- Y축: 온도 (차가움 ↔ 뜨거움)

이 공간에서 각 음료는 좌표(벡터)를 가집니다:

- **아이스 아메리카노**: [-0.8, -0.9] (가볍고 차가움)
- **에스프레소**: [0.8, 0.9] (무겁고 뜨거움)
- **따뜻한 라떼**: [0.7, 0.8] (무겁고 뜨거움)

이 벡터 공간에서 '에스프레소'와 '따뜻한 라떼'는 서로 가까운 위치에 존재합니다. 즉, 벡터 간의 거리가 가까울수록 의미적으로 유사하다고 판단할 수 있습니다. 반면 '아이스 아메리카노'는 이들과 멀리 떨어져 있습니다. 실제 임베딩 모델(OpenAI의 text-embedding-3-large 등)은 2 차원이 아닌 1536 차원, 3072 차원의 초고차원 공간을 사용하여 단어의 미세한 뉘앙스까지 포착합니다.

코사인 유사도는 문장의 길이와 무관하게 '의미의 방향성'을 측정합니다

Why Cosine Similarity?
텍스트의 의미는 벡터의 크기(문장 길이)가 아닌 **방향**에 담겨 있습니다.
코사인 유사도는 두 벡터 사이의 각도(θ)를 측정하여 방향의 일치도를 계산합니다.

similarity = $\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$

지표	측정 대상	특징
유클리드 거리	물리적 거리	벡터 크기(문장 길이)에 민감
코사인 유사도	방향 (각도)	벡터 크기에 무관, 텍스트 검색에 최적

수식으로 표현하면 다음과 같습니다:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

- 두 벡터가 완전히 동일한 방향을 가리키면 각도는 0° 이고 코사인 값은 1 입니다 (완전 일치).
- 두 벡터가 90° 로 직교하면 코사인 값은 0 입니다 (관련 없음).
- 두 벡터가 반대 방향이면 코사인 값은 -1 입니다 (반대 의미).

'모카'와 '우유'의 예시를 통해 계산 과정을 복습해 봅시다.

- $V_{mocha} = [0.584, 0.504]$
- $V_{milk} = [1.037, 0.504]$

이 두 벡터의 내적을 각 벡터의 크기 곱으로 나누면 약 0.966 이라는 높은 유사도 값이 나옵니다. 이는 모카와 우유가 성분상으로(혹은 이 가상의 벡터 공간에서) 매우 밀접한 관계임을 수학적으로 증명하는 것입니다.¹

1.5. 벡터 데이터베이스와 리트리버

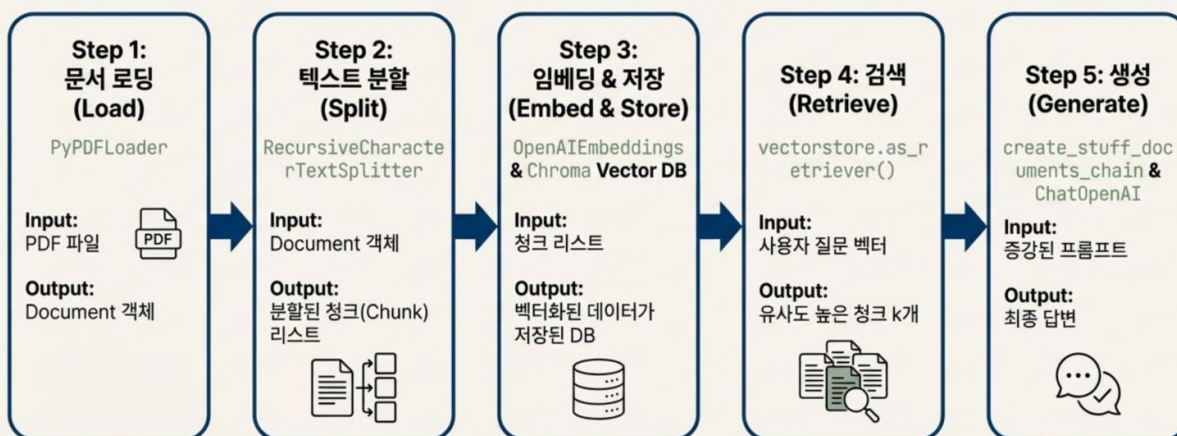
이렇게 변환된 수만 개의 벡터들을 효율적으로 저장하고 검색하기 위해 **벡터 데이터베이스(Vector DB)**가 필요합니다. Chroma DB, Pinecone, Weaviate 등이 대표적입니다. 벡터 DB 는 고차원 공간에서 가장 가까운 이웃(Nearest Neighbor)을 매우 빠르게 찾아내는 알고리즘(HNSW 등)을 내장하고 있습니다. **리트리버(Retriever)**는 이 벡터 DB 를 사용하여 사용자의 질문 벡터와 가장 유사도가 높은 k 개의 청크를 가져오는 역할을 수행합니다.

2. RAG 기반 챗봇 구현하기: 실전 파이프라인 구축

소스코드

sec01/rag_practice.ipynb

LangChain 프레임워크를 활용한 RAG 챗봇 구현 파이프라인



이제 이론적 배경을 바탕으로, Python 과 LangChain 프레임워크를 사용하여 실제 RAG 시스템을 구축하는 과정을 단계별로 상세히 분석하겠습니다. 우리는 서울시와 뉴욕시의 도시 계획 문서를 기반으로 답변하는 챗봇을 만들 것입니다.

2.1. 환경 설정 및 라이브러리 준비

RAG 파이프라인은 여러 구성 요소의 결합으로 이루어집니다. 이를 위해 LangChain 생태계의 다양한 라이브러리가 필요합니다.

필수 라이브러리 설치

```
!pip install langchain_community langchain_openai langchain_chroma pypdf
```

```
%pip install PyMuPDF pypdf langchain langchain_community
```

- langchain_community: PDF 로더 등 다양한 도구 모음입니다.
- langchain_chroma: 벡터 데이터베이스인 Chroma 와의 인터페이스를 제공합니다. 설치 시 numpy 의존성 문제로 인해 C++ 빌드 도구가 필요할 수 있는데, 이는 벡터 연산이 고성능 수치 계산을 요구하기 때문입니다.
- pypdf: PDF 파일에서 텍스트를 추출하는 파서입니다.

2.2. 문서 로딩: 비정형 데이터의 텍스트화

현실 세계의 데이터는 대부분 PDF 와 같은 비정형 형식으로 존재합니다. PyPDFLoader 를 사용하여 이를 처리 가능한 텍스트 객체(Document)로 변환합니다.

```
from langchain_community.document_loaders import PyPDFLoader
```

```
# 뉴욕시 전략 계획 문서 로드
```

```
loader_nyc = PyPDFLoader('./data/OneNYC_2050_Strategic_Plan.pdf')
```

```
data_nyc = loader_nyc.load()
```

```
# 서울시 2040 기본계획 문서 로드
```

```
loader_seoul = PyPDFLoader('./data/2040_seoul_plan.pdf')
```

```
data_seoul = loader_seoul.load()
```

이 단계에서 생성된 data_nyc 는 페이지 단위로 분리된 Document 객체의 리스트입니다. 각 객체는 page_content(본문 텍스트)와 metadata(소스 파일명, 페이지 번호 등)를 속성으로 가집니다. 이 메타데이터는 나중에 챗봇이 답변의 출처를 제시할 때 결정적인 역할을 합니다.

3. 텍스트 분할(Splitting): 재귀적 청킹 전략

앞서 논의한 청킹 이론을 실제 코드로 구현합니다. RecursiveCharacterTextSplitter 는 가장 범용적으로 사용되는 강력한 분할기입니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(
```

```
chunk_size=1000, # 청크 당 약 1000 자
chunk_overlap=100 # 100 자의 오버랩 구간 설정
)

all_splits = text_splitter.split_documents(data_nyc + data_seoul)
```

심층 분석: 왜 재귀적(Recursive)인가?

단순히 글자 수로만 자르면 단어가 반으로 갈리거나 문맥이 파괴될 수 있습니다. 재귀적 분할기는 먼저 문단(WnWn) 단위로 분할을 시도합니다. 만약 문단이 1000 자를 넘으면, 다음으로 문장(Wn) 단위로, 그래도 크다면 단어(공백) 단위로 분할을 시도합니다. 즉, 가장 자연스러운 의미 단위를 유지하려고 노력하면서 크기 제한을 맞추는 방식입니다.

코드 레벨에서 본 RAG 파이프라인 구축: 로딩, 분할, 그리고 저장

1. Loading & Splitting

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader('./data/2040_seoul_plan.pdf')
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
all_splits = text_splitter.split_documents(loader.load())
```

2. Embedding & Storing

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model='text-embedding-3-large')
vectorstore = Chroma.from_documents(
    documents=all_splits,
    embedding=embedding_model,
    persist_directory='./chroma_db' # 디스크에 저장하여 재사용
)
```

Why 'text-embedding-3-large'?

모델	성능(MTEB)	비용(\$/1M 토큰)
...-large	64.6% (최상)	\$0.13
...-small	62.3%	\$0.02

선택 이유: 한국어 문서의 미묘한 뉘앙스 포착을 위한 최고 성능 모델 채택.

서울시 문서가 페이지 단위로 내용이 딱딱 끊어지지 않아 단순 로딩 시 문장이 유실되는 현상이 발생했다는 것입니다. 이를 해결하기 위해 실습에서는 앞 페이지의 끝부분과 뒷 페이지의 앞부분을 강제로 연결하는 전처리 코드를 작성하여 오버랩을 수동으로 구현했습니다. 이는 RAG 구축 시 데이터의 특성에 따른 전처리(Preprocessing)가 얼마나 중요한지를 보여주는 실전적인 교훈입니다.

2.4. 임베딩 및 벡터 저장소 구축

이제 분할된 텍스트 조각들을 벡터로 변환하여 Chroma DB 에 저장합니다.

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

# 임베딩 모델 초기화 (text-embedding-3-large 권장)
embedding_model = OpenAIEmbeddings(model='text-embedding-3-large')

# 벡터 저장소 생성 및 저장
persist_directory = './chroma_db_store'
vectorstore = Chroma.from_documents(
```

```
documents=all_splits,  
embedding=embedding_model,  
persist_directory=persist_directory  
)
```

여기서 `persist_directory` 를 설정하는 것이 중요합니다. 이를 설정하지 않으면 데이터베이스가 메모리(RAM)에만 존재하여 프로그램 종료 시 사라집니다. 디스크에 저장(Persistence)함으로써 한번 임베딩한 데이터를 재사용할 수 있어 비용과 시간을 절약할 수 있습니다.

표 1: OpenAI 임베딩 모델 비교

모델명	성능 (MTEB 점수)	비용 (1M 토큰 당)	특징
text-embedding-3-large	64.6% (최상)	\$0.13	고성능, 다국어 처리 우수, 3072 차원
text-embedding-3-small	62.3%	\$0.02	가성비 우수, 빠른 속도
text-embedding-ada-002	61.0%	\$0.10	구형 모델, 현재는 비추천

한국어 문서의 미묘한 뉘앙스를 포착하기 위해 text-embedding-3-large 를 사용합니다.

2.5. 검색 및 답변 생성 체인 (Chain) 구성

이제 "보좌관"과 "장관"을 연결할 차례입니다.

```
# 리트리버 설정: 유사도 상위 3 개 문서 추출
```

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

```
# 프롬프트 템플릿 정의
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
prompt = ChatPromptTemplate.from_messages([
```

```
    ("system", "당신은 도시 정책 전문가입니다. 아래의 context 만을 기반으로  
답변하세요:\n\n{context}"),
```

```
    ("human", "{input}")
```

```
])
```

```
# 체인 생성 (Retrieve -> Prompt -> LLM)
```

```
from langchain.chains.combine_documents import create_stuff_documents_chain
```

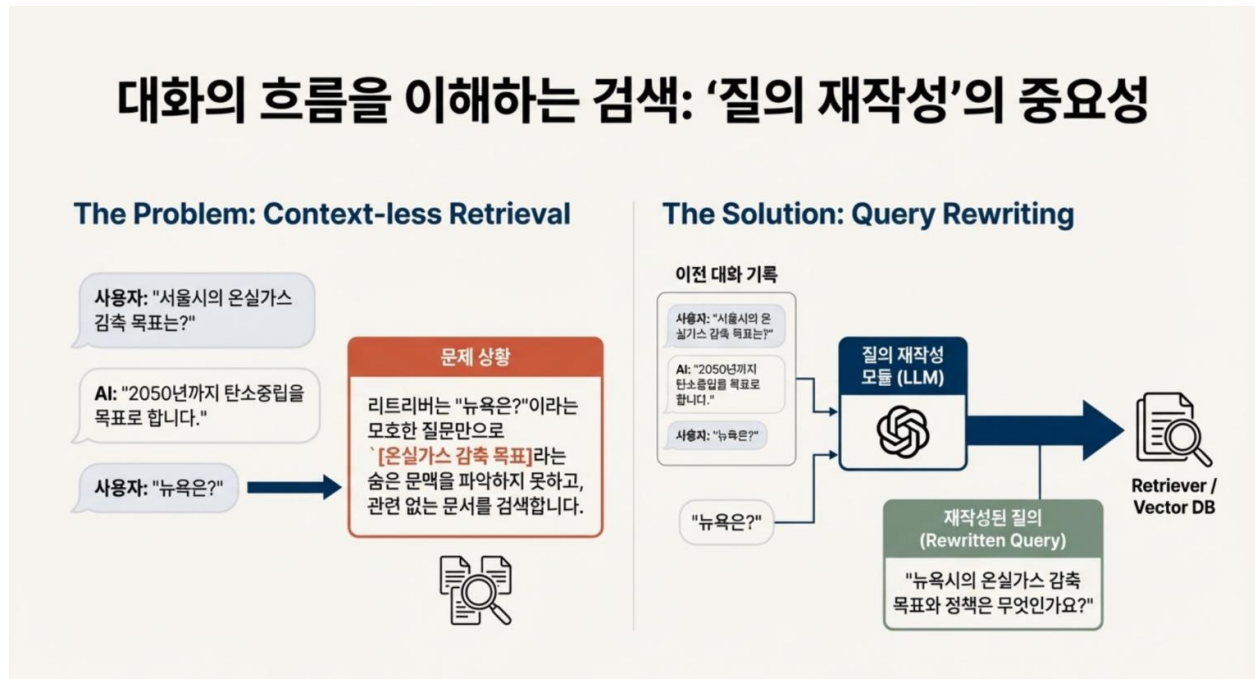
```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4o")
```

```
rag_chain = create_stuff_documents_chain(llm, prompt)
```

여기서 {context} 변수가 핵심입니다. 사용자가 질문을 하면, 리트리버가 찾아온 문서 조각들이 이 {context} 자리에 자동으로 채워집니다(Stuffing). 시스템 프롬프트에 *****아래의 context 만을 기반으로 답변하세요*****라는 제약 조건을 걸어줌으로써 모델의 외부 지식 사용을 억제하고 환각을 방지합니다.

2.6. 질의 확장 (Query Augmentation): 문맥을 이해하는 검색



RAG 의 가장 큰 약점 중 하나는 대화의 문맥을 잃어버리는 것입니다.

- 사용자: "서울시의 온실가스 감축 목표는?"
- AI: "2050 년까지 탄소중립을 목표로 합니다."
- 사용자: "뉴욕은?"

이때 "뉴욕은?"이라는 질문을 그대로 검색하면, "뉴욕"이라는 단어가 들어간 무관한 문서들(관광, 교통 등)이 검색될 수 있습니다. 리트리버는 앞선 대화 내용을 모르기 때문입니다. 이를 해결하기 위해 **질의 확장(Query Augmentation)** 또는 **질의 재작성(Query Rewriting)** 기술을 사용합니다.

질의 재작성 프롬프트

```
rewrite_prompt = ChatPromptTemplate.from_messages([  
    ("system", "대화 기록을 바탕으로 사용자의 모호한 질문을 구체적이고 독립적인 질문으로 재작성하세요."),
```

```
("human", "이전 대화: {chat_history}\n 현재 질문: {input}")  
])
```

재작성 체인 실행

```
rewritten_query = rewrite_chain.invoke({"chat_history": history, "input": "뉴욕은?"})
```

결과: "뉴욕시의 온실가스 감축 목표와 정책은 무엇인가요?"

이렇게 재작성된 명확한 질문을 리트리버에 전달함으로써, 문맥에 맞는 정확한 문서를 찾아낼 수 있습니다. 이는 HyDE(Hypothetical Document Embeddings)와 같은 고급 RAG 기법의 기초가 되는 중요한 단계입니다.

3. 스트림릿(Streamlit)으로 완성하는 인터랙티브 챗봇

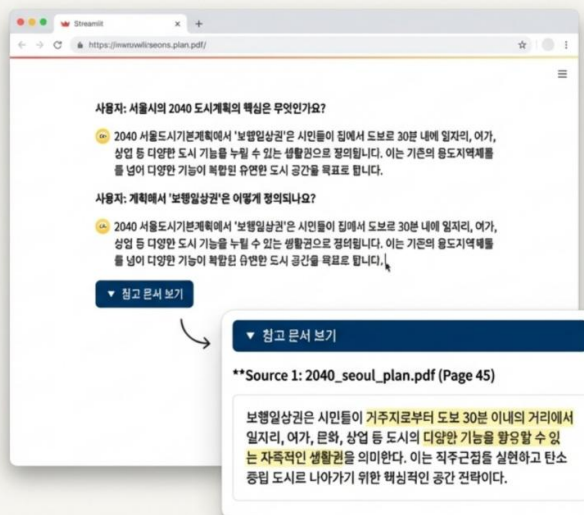
신뢰를 구축하는 인터페이스: Streamlit을 활용한 UX 최적화

1. 스트리밍 응답 (Streaming Response)

- What: `st.write\_stream`을 사용하여 LLM이 생성하는 답변을 타자 치듯 실시간으로 표시.
- Why: 사용자의 대기 시간을 줄여 체감 속도를 획기적으로 개선.

2. 출처 표기 (Citation & Verifiability)

- What: `st.expander` 내부에 답변 생성에 사용된 원문 청크와 출처(파일명, 페이지 번호)를 명확히 제시.
- Why: 사용자가 AI 답변의 근거를 직접 확인하고 검증할 수 있게 하여 시스템의 투명성과 신뢰도를 높임.



콘솔 창에서의 실행은 개발자에게만 유용합니다. 실제 사용자를 위해서는 직관적인 그래픽 인터페이스(GUI)가 필요합니다. 파이썬 기반의 웹 앱 프레임워크인 **Streamlit** 은 이러한 프로토타이핑에 최적화되어 있습니다.

3.1. 상태 관리 (Session State): 기억을 가진 웹 앱

Streamlit 은 사용자가 버튼을 누르거나 입력을 할 때마다 스크립트 전체를 다시 실행하는 구조를 가집니다. 따라서 변수에 저장된 대화 기록이 초기화될 수 있습니다. 이를 방지하기 위해 `st.session_state` 라는 영구 저장소를 사용합니다.

```
import streamlit as st
```

```
# 대화 기록 저장소 초기화
```

```
if "messages" not in st.session_state:
```

```
    st.session_state["messages"] =
```

이 `session_state` 에 사용자의 질문과 AI 의 답변을 순차적으로 저장(append)하고, 앱이 다시 실행될 때마다 이 리스트를 순회하며 화면에 뿌려줌으로써 대화가 이어지는 듯한 경험을 제공합니다.

3.2. 사용자 경험(UX) 최적화: 스트리밍과 피드백

LLM의 답변 생성을 기다리는 시간은 사용자에게 지루할 수 있습니다. `st.write_stream`을 사용하여 타자기를 치듯 글자가 생성되는 즉시 화면에 보여주는 **스트리밍(Streaming)** 기능을 구현하면 체감 속도를 획기적으로 개선할 수 있습니다.

또한, RAG 시스템의 핵심 가치인 신뢰성을 시각적으로 보여주기 위해 **출처 표기(Citation)** UI를 구현합니다.

문서 출처 표시 UI

with `st.expander("참고 문서 보기")`:

for `doc` in `retrieved_docs`:

`st.write(f"***출처:** {doc.metadata['source']} (페이지: {doc.metadata['page']})")`

`st.info(doc.page_content[:200] + "...")`

`st.expander`를 사용하면 평소에는 숨겨져 있다가 사용자가 원할 때만 펼쳐볼 수 있어 깔끔한 UI를 유지하면서도 투명성을 보장할 수 있습니다. 사용자는 AI의 답변이 실제 문서의 어느 부분에서 왔는지 직접 검증할 수 있게 됩니다.

심화 연구 및 향후 전망 (2025+)

RAG 기술의 진화: 단순 검색을 넘어 자율적 추론으로



지금까지 내용에서 다룬 내용은 'Standard RAG'입니다. 하지만 2025 년 현재 RAG 기술은 더욱 정교하게 진화하고 있습니다.

1. GraphRAG: 관계를 이해하는 검색

벡터 검색은 키워드나 의미적 유사성에는 강하지만, "A 와 B 의 공통점은 무엇인가?"와 같이 여러 문서에 흩어진 정보를 종합해야 하는 질문에는 취약합니다. **GraphRAG** 는 문서를 지식 그래프(Knowledge Graph) 형태로 구조화하여 엔티티 간의 관계를 추적함으로써 이러한 'Multi-hop' 추론 능력을 획기적으로 개선합니다.

2. Agentic RAG: 자율적인 검색 요원

기존 RAG 가 검색->생성의 선형적 구조라면, **Agentic RAG** 는 AI 가 스스로 판단합니다. "이 질문은 문서 검색이 필요 없어"라고 판단하면 검색을 생략하거나, "검색 결과가 부족해, 웹 검색을 추가로 해야겠어"라고 판단하여 도구(Tool)를 사용합니다. 이는 RAG 를 단순한 파이프라인이 아닌 능동적인 문제 해결 에이전트로 진화시킵니다.

3. 평가와 최적화 (RAGAS)

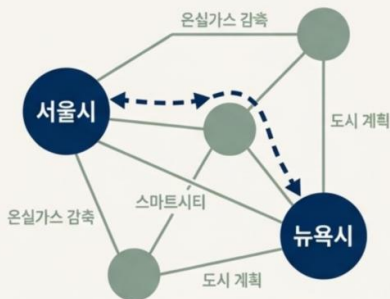
RAG 시스템을 구축한 후에는 "얼마나 잘 작동하는가?"를 측정해야 합니다. **RAGAS(RAG Assessment)** 프레임워크는 '답변이 문서에 근거했는가(Faithfulness)', '답변이 질문과 관련

있는가(Answer Relevance)', '검색된 문서가 적절했는가(Context Precision)' 등의 지표를 통해 RAG 시스템을 정량적으로 평가합니다.

심화 연구: 관계를 이해하고 스스로 판단하는 차세대 RAG

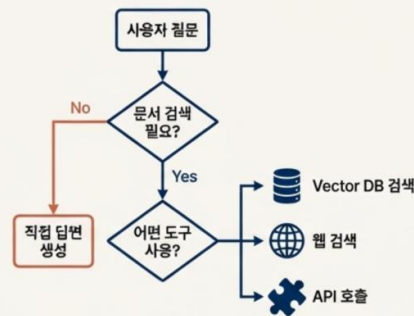
1. GraphRAG: 관계를 이해하는 검색

문서를 지식 그래프(Knowledge Graph)로 구조화. 엔티티 간의 관계를 추적하여 "A와 B의 공통점은?" 과 같은 복잡한 'Multi-hop' 추론 수행.



2. Agentic RAG: 자율적인 검색 요원

AI 에이전트가 스스로 질문을 분석하여 검색 필요 여부, 추가 검색(웹 검색 등) 필요 여부를 판단하고 적절한 도구(Tool)를 자율적으로 사용.



3. RAG 시스템 평가 (RAGAS Framework)

구축된 시스템의 성능을 정량적으로 측정하는 것이 중요.



Faithfulness
(답변의 근거 여부)



Answer Relevance
(답변의 관련성)



Context Precision
(검색의 정확성)

당신은 이제, 천재 장관에게 완벽한 보좌관을 붙여주었습니다.



RAG는 LLM의 환각 문제를 해결하고, 최신 정보를 반영하며, 근거를 제시하는 가장 강력한 아키텍처입니다. 이는 데이터를 신뢰할 수 있는 지식 서비스로 변환하는 AI 엔지니어링의 정수입니다.

Call to Action:

이 시스템을 기반으로 여러분만의 도메인 지식(법률, 의료, 사내 규정)을 결합하여 세상에 없던 가치를 창출하는 챗봇을 만들어 보시기 바랍니다.

데이터 테이블 및 비교 자료

표 2: 거리 측정 지표 비교 (Distance Metrics)

지표 (Metric)	수식 (Formula)	특징 및 장단점	최적 사례
유클리드 거리 (Euclidean)	$\sqrt{\sum (A_i - B_i)^2}$	물리적 거리를 측정. 벡터의 크기(문장 길이 등)에 민감하여 텍스트 유사도 측정 시 왜곡될 수 있음.	이미지 분류, 위치 데이터
코사인 유사도 (Cosine)	$\frac{A \cdot B}{\ A\ \ B\ }$	**벡터의 방향(각도)**을 측정. 벡터 크기에 영향을 받지 않아 문장 길이가 달라도 의미가 같으면 유사하게 판단함.	텍스트 검색, 문서 분류, RAG
내적 (Dot Product)	$\sum A_i B_i$	연산 속도가 빠름. 벡터가 정규화(Normalized)되어 있다면 코사인 유사도와 동일한 결과를 냄.	추천 시스템, 정규화된 임베딩

표 3: 청킹 전략 비교 (Chunking Strategies)

전략	설명	장점	단점
고정 크기 (Fixed Size)	정해진 글자/토큰 수로 무조건 자름	구현이 쉽고 빠름.	문장이나 문단이 중간에 잘려 의미가 훼손될 수 있음.
재귀적 문자 (Recursive)	문단 -> 문장 -> 단어 순으로 구분자를 찾아 자름	문맥과 의미 단위를 최대한 보존함.	고정 크기보다 연산 비용이 약간 더 높음.
의미론적 (Semantic)	임베딩 유사도가 급격히 변하는 지점에서 자름	주제가 바뀌는 지점을 정확히 포착하여 가장 높은 품질의 청크 생성.	모든 문장을 임베딩해야 하므로 비용과 시간이 많이 들음.
에이전트 기반 (Agentic)	LLM 에게 문서를 주고 "의미 단위로 요약/분할해줘"라고 요청	인간 수준의 이해도를 바탕으로 분할.	속도가 매우 느리고 토큰 비용이 매우 높음.

표 4: RAG 기술의 진화 단계

구분	Standard RAG (2023)	Advanced RAG (2024)	Graph / Agentic RAG (2025+)
검색 방식	단순 벡터 유사도 검색	하이브리드 검색 (키워드 + 벡터), 리랭킹(Re-ranking)	지식 그래프 탐색, 멀티홉(Multi-hop) 추론
질의 처리	사용자 질문 그대로 사용	질의 재작성, HyDE (가상 답변 생성)	능동적 계획(Planning), 도구 사용
정확도	단순 사실 검색에 유리	문맥 이해도 향상	복잡한 관계 추론 및 종합적 답변 가능